CÓDIGOCLARO · MATERIAL DE ESTUDIO

Python desde cero

Cuaderno DAM/DAW · Python 3.14 · ejemplos con biblioteca estándar, sin dependencias externas.

Intérprete y punto de entrada

Objetivo: Ejecutar un script desde consola sin que se imprima al importarlo desde otro módulo.

Python ejecuta instrucciones en orden y delimita bloques mediante indentación. El guard de __main__ separa una demostración ejecutable de las definiciones reutilizables; al importar, el código dentro del guard no se ejecuta. Anota el intérprete usado y el directorio de trabajo para reproducir un problema. Evita que un fichero del proyecto tenga el mismo nombre que una biblioteca estándar que necesita importar.

Código · Python 3.14

```
def saludar(nombre):
    return f"Hola, {nombre}"

if __name__ == "__main__":
    print(saludar("Ada"))
```

El guard protege efectos como imprimir, leer argumentos o empezar una demostración cuando se importa el módulo. Mantén las funciones y clases al nivel superior para que otro archivo pueda reutilizarlas. Ejecuta el mismo fichero con el intérprete elegido para el entorno y registra la versión, porque una sintaxis reciente puede no existir en una instalación antigua. Si un programa funciona desde una carpeta y falla desde otra, revisa primero el directorio de trabajo y las rutas relativas. El guard protege efectos como imprimir, leer argumentos o empezar una demostración cuando se importa el módulo. Mantén las funciones y clases al nivel superior para que otro archivo pueda reutilizarlas. Ejecuta el mismo fichero con el intérprete elegido para el entorno y registra la versión, porque una sintaxis reciente puede no existir en una instalación antigua. Si un programa funciona desde una carpeta y falla desde otra, revisa primero el directorio de trabajo y las rutas relativas.

Práctica: Importa el módulo desde principal.py. ¿Debe imprimirse el saludo automáticamente?

> Respuesta razonada: No. Al importarse, __name__ contiene el nombre del módulo, no __main__. La función se puede reutilizar sin que el import tenga un efecto de impresión.

Documentación consultada: Tutorial oficial Python 3.14

Tipos, conversiones y dinero

Objetivo: Convertir una entrada de texto y conservar un importe decimal exacto.

input devuelve str aunque la persona escriba números. Convierte de forma explícita y captura ValueError cuando la representación no sea válida. float usa coma flotante binaria; para un importe exacto, construye Decimal desde texto y define la regla de redondeo que usa la aplicación.

Código · Python 3.14

```
from decimal import Decimal
precio = Decimal("19.95")
unidades = int("3")
total = precio * unidades
print(total)  # 59.85
```

input siempre entrega texto; la conversión a int, float o Decimal es una decisión explícita que puede fallar con ValueError. Para cálculos comerciales, crea Decimal desde la cadena escrita y mantén tipos compatibles durante toda la operación. El formato de salida no define por sí mismo una regla de redondeo: acuerda escala y modo con el dominio. Comprueba también cero unidades y datos con separador decimal inesperado antes de presentar un total. input siempre entrega texto; la conversión a int, float o Decimal es una decisión explícita que puede fallar con ValueError. Para cálculos comerciales, crea Decimal desde la cadena escrita y mantén tipos compatibles durante toda la operación. El formato de salida no define por sí mismo una regla de redondeo: acuerda escala y modo con el dominio. Comprueba también cero unidades y datos con separador decimal inesperado antes de presentar un total.

Práctica: ¿Por qué no conviene crear Decimal a partir de float 19.95?

> Respuesta razonada: El float ya contiene una aproximación binaria al convertirse. Decimal desde la cadena conserva el importe decimal escrito; el redondeo debe definirse aparte según la regla contable.

Documentación consultada: Tutorial oficial Python 3.14 · Documentación oficial Python 3.14

Condiciones y validación

Objetivo: Clasificar una nota válida, aprobada o suspensa sin admitir valores fuera de escala.

if y elif seleccionan una rama. Valida primero el dominio y después evalúa el aprobado; de lo contrario, 12 podría aceptarse por ser mayor que cinco. Incluye casos frontera 0, 5 y 10 y separa entrada inválida de resultado suspenso.

Código · Python 3.14

```
def clasificar(nota):
    if not 0 <= nota <= 10:
        return "inválida"
    if nota >= 5:
        return "aprobada"
    return "suspensa"
```

Las ramas se evalúan en orden: primero valida que la nota esté dentro del dominio y luego decide si alcanza el aprobado. Así un 12 no se clasifica como aprobada solo porque sea mayor que cinco. Convierte input a número antes de invocar la función y maneja aparte un texto que no pueda convertirse. Prueba -1, 0, 5, 10 y 12 para cubrir fuera de rango, ambos extremos y el umbral inclusivo. Las ramas se evalúan en orden: primero valida que la nota esté dentro del dominio y luego decide si alcanza el aprobado. Así un 12 no se clasifica como aprobada solo porque sea mayor que cinco. Convierte input a número antes de invocar la función y maneja aparte un texto que no pueda convertirse. Prueba -1, 0, 5, 10 y 12 para cubrir fuera de rango, ambos extremos y el umbral inclusivo.

Práctica: Calcula las respuestas para -1, 0, 5 y 10. ¿Qué ocurre con 12?

> Respuesta razonada: Las salidas son inválida, suspensa, aprobada y aprobada; 12 también es inválida. Validar el rango antes del aprobado mantiene el contrato explícito.

Documentación consultada: Tutorial oficial Python 3.14

Bucles y terminación

Objetivo: Sumar cantidades con un recorrido cuyo estado y final sean fáciles de verificar.

for recorre elementos; range genera enteros hasta un extremo que no incluye. El acumulador comienza en cero y cambia en cada iteración. Un while necesita una condición de salida y un avance demostrable. Prueba la colección vacía y el elemento final para detectar límites incorrectos.

Código · Python 3.14

```
pendientes = [2, 0, 3]
total = 0
for cantidad in pendientes:
    total += cantidad
print(total)  # 5
```

range(3) produce 0, 1 y 2; el límite superior queda excluido. En un bucle con índices, esa propiedad coincide con las posiciones válidas de una lista de tres elementos. En while, identifica qué variable cambia y qué condición acaba siendo falsa; si ninguna cambia, el bucle puede no terminar. Usa una lista vacía y una con un elemento para confirmar el valor inicial del acumulador y que no hay una vuelta extra. range(3) produce 0, 1 y 2; el límite superior queda excluido. En un bucle con índices, esa propiedad coincide con las posiciones válidas de una lista de tres elementos. En while, identifica qué variable cambia y qué condición acaba siendo falsa; si ninguna cambia, el bucle puede no terminar. Usa una lista vacía y una con un elemento para confirmar el valor inicial del acumulador y que no hay una vuelta extra.

Práctica: Escribe el recorrido equivalente con índices. ¿Qué valores produce range(len(pendientes))?

> Respuesta razonada: Produce 0, 1 y 2, uno por cada posición. El rango excluye el 3; añadir uno al límite intentaría acceder a un índice inexistente.

Documentación consultada: Tutorial oficial Python 3.14

Listas, tuplas, conjuntos y diccionarios

Objetivo: Elegir una estructura para mantener orden y otra para evitar códigos duplicados.

list conserva secuencia mutable; tuple representa una secuencia que no se modifica en el sitio; set modela pertenencia sin duplicados; dict relaciona claves con valores. Copiar una lista anidada es superficial: los objetos internos pueden seguir compartidos. El orden de presentación debe fijarse explícitamente cuando sea requisito.

Código · Python 3.14

```
modulos = ["Programación", "Bases de datos"]
codigos_unicos = set(["PRG", "PRG", "BBDD"])
notas = {"PRG": 8, "BBDD": 7}
print(notas.get("PRG"))
```

list y tuple conservan secuencia; la lista se modifica y la tupla expresa una colección que no se cambia en el sitio. set elimina duplicados, pero no ofrece un orden de presentación para un informe. dict asocia claves únicas con valores y conserva el orden de inserción en las versiones actuales de Python. La copia de una lista es superficial si contiene otras listas: prueba si modificar un elemento interno afecta también a la estructura original. list y tuple conservan secuencia; la lista se modifica y la tupla expresa una colección que no se cambia en el sitio. set elimina duplicados, pero no ofrece un orden de presentación para un informe. dict asocia claves únicas con valores y conserva el orden de inserción en las versiones actuales de Python. La copia de una lista es superficial si contiene otras listas: prueba si modificar un elemento interno afecta también a la estructura original.

Práctica: ¿Qué usarías para conservar notas en orden y qué para impedir códigos duplicados?

> Respuesta razonada: Una lista conserva el orden de notas; un conjunto elimina duplicados de códigos. Si el orden de códigos también importa, usa dict como conjunto ordenado en Python moderno y documenta esa decisión.

Documentación consultada: Tutorial oficial Python 3.14

Funciones y valores por defecto

Objetivo: Calcular una tasa de IVA y rechazar precios negativos con una función reutilizable.

Los parámetros nombrados aclaran la llamada. Los valores por defecto se evalúan al definir la función; evita una lista mutable por defecto cuando cada llamada deba recibir estado nuevo. Una función reutilizable devuelve valor y declara errores, en lugar de imprimir como efecto lateral. Para decimales comerciales, usa Decimal consistentemente.

Código · Python 3.14

```
from decimal import Decimal
def calcular_iva(precio, tipo=Decimal("0.21")):
    if precio < 0:
        raise ValueError("Precio negativo")
    return precio * (1 + tipo)
```

El valor por defecto se evalúa una vez al definir la función. Si se usa una lista mutable, llamadas distintas pueden compartirla accidentalmente; usa None y crea una nueva lista dentro cuando cada llamada necesite estado independiente. Devuelve el resultado para que otra función pueda utilizarlo y deja la impresión a la capa de presentación. El precio y la tasa deben tener tipos compatibles, en este ejemplo Decimal, para no mezclar precisión binaria y decimal. El valor por defecto se evalúa una vez al definir la función. Si se usa una lista mutable, llamadas distintas pueden compartirla accidentalmente; usa None y crea una nueva lista dentro cuando cada llamada necesite estado independiente. Devuelve el resultado para que otra función pueda utilizarlo y deja la impresión a la capa de presentación. El precio y la tasa deben tener tipos compatibles, en este ejemplo Decimal, para no mezclar precisión binaria y decimal.

Práctica: ¿Qué excepción produce el precio -1 y qué devuelve 100 con tipo 0.10?

> Respuesta razonada: El precio negativo lanza ValueError; 100 con tasa Decimal 0.10 devuelve 110.00. Mantén ambos operandos Decimal para no mezclar precisión decimal y binaria.

Documentación consultada: Tutorial oficial Python 3.14

Módulos y entorno virtual

Objetivo: Aislar dependencias y comprobar qué intérprete ejecuta un proyecto de aula.

Un módulo es un fichero importable y un paquete agrupa módulos relacionados. El entorno virtual separa dependencias por proyecto; la biblioteca estándar no requiere pip. Usa el mismo intérprete para instalar y ejecutar paquetes, y registra versiones externas para que el equipo reproduzca el entorno.

Código · Python 3.14

```
python -m venv .venv
# PowerShell:
.\.venv\Scripts\Activate.ps1
python --version
python -m pip --version
```

un módulo suele corresponder a un fichero .py y un paquete organiza varios módulos. Un entorno virtual separa paquetes instalados para un proyecto sin cambiar el Python global. Usa `python -m pip` con el mismo intérprete que ejecuta la aplicación; comprueba la ruta del ejecutable dentro de .venv. No versionas la carpeta .venv: guarda la declaración de dependencias y recrea el entorno cuando otra persona clona el proyecto. Un módulo suele corresponder a un fichero .py y un paquete organiza varios módulos. Un entorno virtual separa paquetes instalados para un proyecto sin cambiar el Python global. Usa `python -m pip` con el mismo intérprete que ejecuta la aplicación; comprueba la ruta del ejecutable dentro de .venv. No versionas la carpeta .venv: guarda la declaración de dependencias y recrea el entorno cuando otra persona clona el proyecto.

Práctica: ¿Qué salida confirma que pip pertenece al entorno activo?

> Respuesta razonada: La ruta mostrada por python -m pip debe apuntar a .venv. Usar pip mediante el propio intérprete reduce el riesgo de instalar en otro Python global.

Documentación consultada: Python 3.14 · venv · Tutorial oficial Python 3.14

Ficheros, JSON y errores

Objetivo: Leer un perfil UTF-8 y distinguir JSON válido de datos que cumplen el contrato.

with cierra el fichero tras leerlo. json.load analiza sintaxis, pero no comprueba el esquema de aplicación; valida tipo raíz y campos después. Distingue fichero ausente, JSON malformado y un documento válido con campos inválidos, porque la respuesta y el diagnóstico son distintos. Fija encoding="utf-8" al abrir texto y conserva el error original para distinguir una codificación o formato inesperado.

Código · Python 3.14 + json

```
import json
from pathlib import Path
with Path("alumno.json").open(encoding="utf-8") as f:
    datos = json.load(f)
if not isinstance(datos, dict) or not isinstance(datos.get("nombre"), str):
    raise ValueError("Perfil inválido")
```

json.load convierte el texto en valores Python y lanza JSONDecodeError si la sintaxis está rota. Un JSON válido puede tener raíz o campos incorrectos. Valida tipos antes de usar get; distingue fichero ausente, formato malformado y contrato incumplido. Abre en UTF-8 y usa pathlib para las rutas. JSONDecodeError incluye posición de línea y columna para localizar un fallo de sintaxis; muéstralo como diagnóstico y evita capturar Exception si no puedes recuperarte.

Práctica: El fichero contiene []. ¿Por qué el parseo puede acabar bien y el perfil ser inválido?

> Respuesta razonada: [] es JSON sintácticamente válido, pero la raíz no es un objeto con nombre. Comprueba el tipo antes de usar get y comunica el incumplimiento de esquema.

Documentación consultada: Biblioteca estándar: json · Biblioteca estándar: pathlib

Clases y invariantes

Objetivo: Modelar una cuenta que rechace saldo inicial negativo y proteja su estado.

Una clase define comportamiento y cada instancia tiene su propio estado. __init__ recibe self y valida la invariante antes de guardar. Dataclass reduce código para datos, pero no añade automáticamente reglas de negocio. Expón operaciones válidas y evita ofrecer asignaciones que permitan romper el estado.

Código · Python 3.14

```
class Cuenta:
    def __init__(self, saldo):
        if saldo < 0:
            raise ValueError("Saldo negativo")
        self._saldo = saldo
    @property
    def saldo(self): return self._saldo
```

__init__ recibe la instancia por self y establece el estado que utilizarán los métodos. Validar saldo antes de guardarlo impide que la cuenta nazca fuera del dominio. Una propiedad de solo lectura permite consultar el saldo sin ofrecer una asignación arbitraria desde fuera. Si se añade ingresar, valida la cantidad antes de actualizar; una operación inválida no debe dejar saldo parcialmente cambiado. Una dataclass ahorra código, pero no valida reglas automáticamente. __init__ recibe la instancia por self y establece el estado que utilizarán los métodos. Validar saldo antes de guardarlo impide que la cuenta nazca fuera del dominio. Una propiedad de solo lectura permite consultar el saldo sin ofrecer una asignación arbitraria desde fuera. Si se añade ingresar, valida la cantidad antes de actualizar; una operación inválida no debe dejar saldo parcialmente cambiado. Una dataclass ahorra código, pero no valida reglas automáticamente.

Práctica: Añade ingresar(cantidad). ¿Debe cambiar el saldo si cantidad es -5?

> Respuesta razonada: No. Lanza ValueError antes de modificarlo. La cuenta mantiene el saldo anterior y conserva la misma regla para cada llamador.

Documentación consultada: Tutorial oficial Python 3.14

Pruebas y depuración

Objetivo: Convertir el contrato de contar pares en pruebas repetibles para entradas límite.

Una prueba prepara una entrada, ejecuta una unidad y compara un resultado observable. Incluye caso habitual, vacío y valores negativos si el dominio los admite. Para depurar, reduce el fallo, observa valores intermedios y prueba una hipótesis cada vez; no ocultes la causa con un resultado inventado. Sigue PEP 8 con nombres descriptivos y formato consistente, pero prioriza que la prueba exprese el contrato, no un detalle interno de implementación.

Código · unittest · Python 3.14

```
import unittest
def contar_pares(xs):
    return sum(1 for n in xs if n % 2 == 0)
class TestPares(unittest.TestCase):
    def test_vacio(self):
        self.assertEqual(contar_pares([]), 0)
```

Cada test debe poder ejecutarse de manera independiente y describir una condición observable. El caso vacío comprueba el valor neutro, el caso normal detecta una suma incorrecta y los negativos confirman el comportamiento de módulo. PEP 8 recomienda nombres legibles, indentación uniforme y líneas manejables; el estilo ayuda a revisar, pero no demuestra corrección. Cuando falle una prueba, reduce los datos y contrasta esperado frente a observado antes de cambiar varias funciones. Cada test debe poder ejecutarse de manera independiente y describir una condición observable. El caso vacío comprueba el valor neutro, el caso normal detecta una suma incorrecta y los negativos confirman el comportamiento de módulo. PEP 8 recomienda nombres legibles, indentación uniforme y líneas manejables; el estilo ayuda a revisar, pero no demuestra corrección. Cuando falle una prueba, reduce los datos y contrasta esperado frente a observado antes de cambiar varias funciones.

Práctica: Añade pruebas para [2, 3, 4] y [-2, -1]. ¿Qué resultados esperas?

> Respuesta razonada: Esperas 2 y 1. El resto módulo dos identifica pares negativos también; el caso vacío fija el valor neutro de la suma.

Documentación consultada: Biblioteca estándar: unittest · PEP 8 · Style Guide for Python Code

Fuentes oficiales

Documentación oficial Python 3.14 · Tutorial oficial Python 3.14 · Biblioteca estándar: json · Biblioteca estándar: pathlib · Biblioteca estándar: unittest · Python 3.14 · venv · PEP 8 · Style Guide for Python Code